TASK 3
Research Results and Data Analysis

Comparative evaluation of machine learning models for URL classification

Zhanna Azatkyzy and Dariyana Gabdylshukir
5 November 2026

1. Research scope and protocol

The broader topic is phishing detection, but the available labels distinguish malicious from benign URLs, without a phishing-specific category. This study therefore evaluates malicious-URL classification. RQ1 compares models; RQ2a compares F1 for 16 versus seven lexical features; RQ2b compares latency by feature set; RQ3 examines the quality-latency trade-off. Logistic Regression with 16 features is the baseline.

From the original 20,000-URL sample, 21 rows failed domain validation. The retained sample contains 19,979 URLs: 3,211 malicious and 16,768 benign, in 10,858 registrable-domain groups. Exact duplicate/conflicting URL handling and domain validation are implemented before splitting. Exclusion may affect representativeness.

Four fixed classifiers (Logistic Regression, Random Forest, RBF SVM and XGBoost) use the same domain-grouped partition within each of ten repeats, seeds 42-51. GroupShuffleSplit holds out 30% of groups; test URL proportions range from 27.27% to 33.99%. This split is not stratified. Saved split indices confirm zero shared domains between training and test within every repeat. Private suffix boundaries use the offline tldextract 5.4.0 snapshot.

One local worker processes one URL per request at scheduled rates of 50, 200 and 500 requests/s. There are 20 warm-up and 1,000 evaluation calls per model, feature set, repeat and rate: 4,800 warm-up and 240,000 evaluation calls. Training and warm-up are excluded. This is a local inference benchmark, without HTTP/network overhead. Numerical libraries are limited to one thread.

2. Processing and uncertainty

The notebook loads the complete raw JSONL (gzip), filters phase=warmup, calculates metrics from confusion counts, aggregates latency and exports figures. For quality, mean and sample SD summarize ten repeat scores. For latency, p50, p95 and p99 are calculated within each repeat, then averaged. The summary p99 is not a pooled percentile. Bootstrap CIs resample ten repeat-level statistics 2,000 times, seed 42, taking the 2.5th and 97.5th percentiles. Repeated splits share observations; intervals are conditional and descriptive. No formal significance test is performed.

3. Classification results

Table 1. Malicious-class F1, sample SD and 95% bootstrap CI over ten domain-disjoint repeats. Recall is the mean repeat-level value. LR full is the baseline. All settings are fixed, without test-set tuning.

Model	Features	F1 mean +/- SD	F1 95% CI	Recall
Logistic Regression	full	0.324 +/- 0.028	[0.308, 0.340]	0.200
Logistic Regression	selected	0.214 +/- 0.019	[0.202, 0.225]	0.122
Random Forest	full	0.590 +/- 0.020	[0.578, 0.601]	0.555
Random Forest	selected	0.367 +/- 0.015	[0.359, 0.377]	0.296
SVM	full	0.393 +/- 0.029	[0.376, 0.410]	0.255
SVM	selected	0.276 +/- 0.024	[0.264, 0.292]	0.165
XGBoost	full	0.602 +/- 0.018	[0.592, 0.613]	0.512
XGBoost	selected	0.370 +/- 0.018	[0.360, 0.381]	0.254

RQ1. XGBoost full has the highest observed mean F1 (0.602), followed by Random Forest (0.590), SVM (0.393) and baseline Logistic Regression (0.324). This ranking describes the recorded dataset and partitions; it is not a statistical significance claim.

RQ2a. Seven features reduce mean F1 for all four models. For example, Random Forest falls from 0.590 to 0.367 and XGBoost from 0.602 to 0.370. Fewer features do not improve classification quality in this experiment.

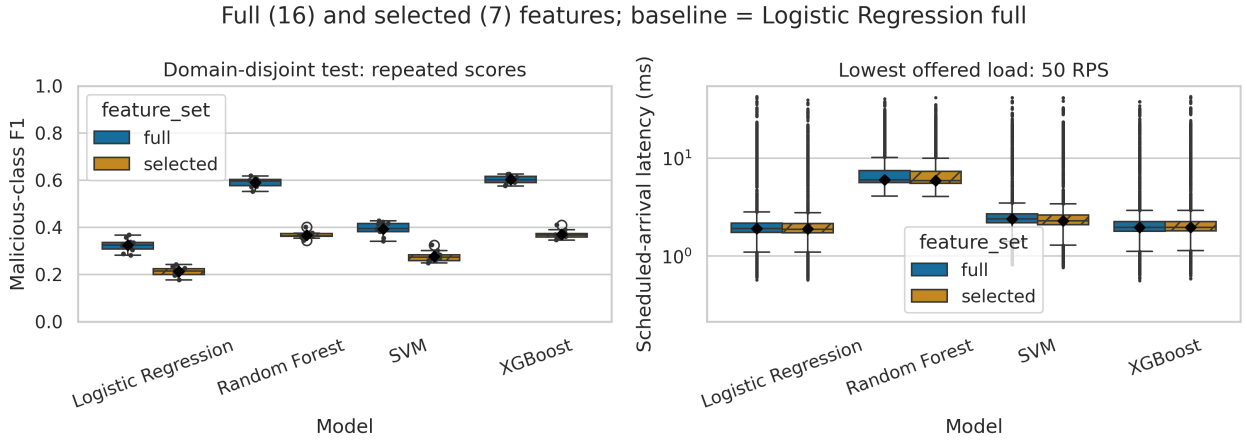


Figure 2. Left: F1 across ten repeats. Right: request latency at 50 requests/s, logarithmic scale. Boxes show request/repeat dispersion as appropriate. Black diamonds and vertical intervals show mean F1 and mean repeat p50 with 95% bootstrap CI. Full/selected sets are distinguished by color and hatching. Vector PDF and 300 DPI PNG are provided.

4. Latency, load and guardrails

Latency is completion minus intended arrival: scheduler lag + queueing + service time. Service includes feature extraction and prediction. Actual submission rate and completion throughput are reported separately. Each finite trial drains its queue after arrivals stop; it does not establish long-term sustainable capacity.

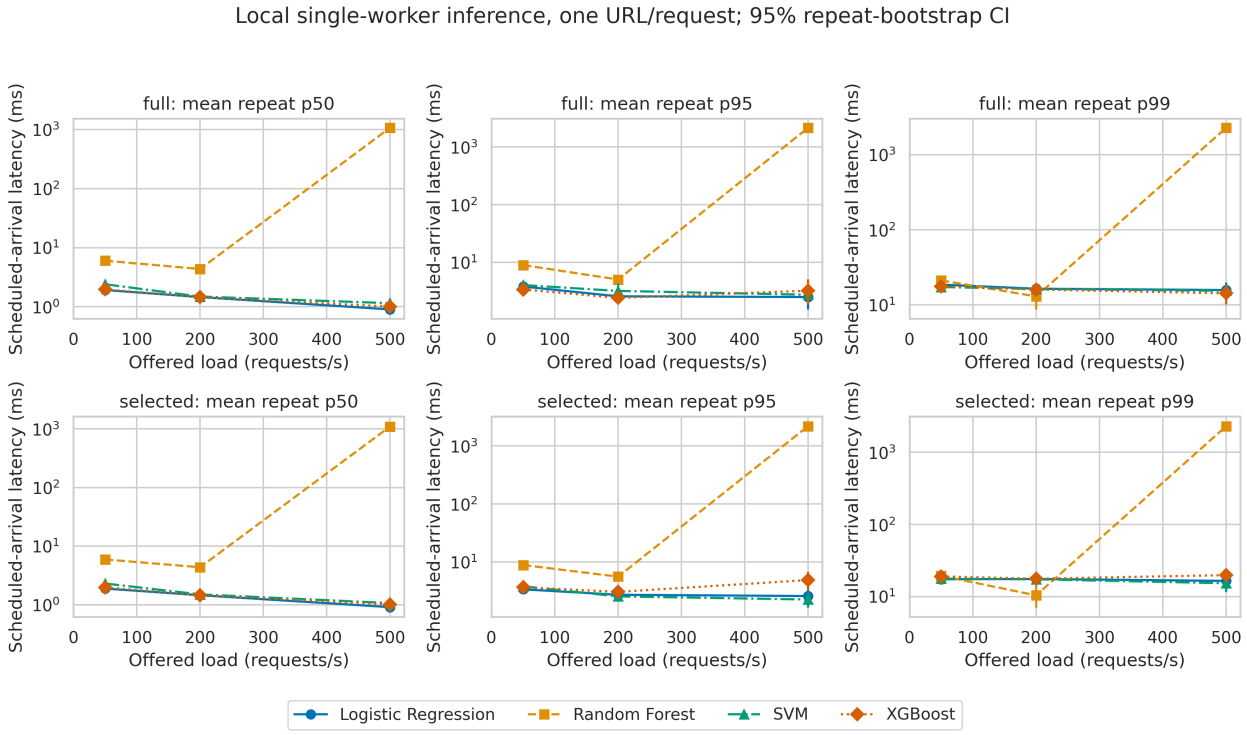


Figure 1. Mean repeat-level p50, p95 and p99 by scheduled request arrival rate. Vertical lines show 95% bootstrap intervals; latency uses a logarithmic axis. Models differ by colorblind-safe colors, line styles and markers.

Model (full)	50 RPS p50 / p99	200 RPS p50 / p99	500 RPS p50 / p99
Logistic Regression	1.92 / 18.64	1.46 / 16.47	0.90 / 15.81
Random Forest	6.02 / 21.26	4.35 / 13.00	1066.17 / 2265.84
SVM	2.40 / 17.35	1.49 / 16.33	1.15 / 15.44
XGBoost	1.97 / 17.58	1.45 / 16.05	1.01 / 14.29

Table 2. Mean repeat-level latency percentiles, milliseconds. Complete values and CIs for both feature sets are in metrics_summary.csv.

RQ2b and RQ3. Random Forest develops a queue at 500 RPS: mean repeat p99 is 2.27-2.28 seconds, with about 232-233 successful completions/s over the trial including queue drain. XGBoost full combines the best observed mean F1 with latency below the specified threshold across the tested loads. Some median latencies decrease as arrival rate increases; scheduling and environment effects are included, so this is not a general causal speedup claim.

All evaluation calls finish without recorded operational errors. No model meets mean recall >=0.80; the best is Random Forest full, 0.555. All configurations meet worst-repeat p99 <=100 ms at 50 and 200 RPS. At 500 RPS, Random Forest fails for both feature sets, with worst-repeat p99 of 2.31-2.42 seconds. No configuration passes every guardrail.

5. ML domain model

The full vector has 16 lexical values: URL and hostname length; counts of dots, hyphens, slashes, question marks, equals signs and digits; HTTPS and at-sign indicators; indicators for login, verify, account, secure, confirm and update. The seven predefined values are URL length, dots, hyphens, slashes, HTTPS, login and verify. Models use predict; SVM probability estimation is disabled. There is no common probability threshold across all four models.

For Logistic Regression, the data loss is binary log-loss:
$\mathbf{L_{data} = -(1/n)\ sum_{i=1..n}\ [y_i\ log\ p_i + (1-y_i)\ log(1-p_i)]}$.
L2 regularisation also penalizes weights. SVC uses regularised hinge loss, Random Forest Gini split impurity, and XGBoost a binary logistic objective with tree regularisation. Formal LaTeX definitions and the notation table are in docs/domain_model.md.

Precision = TP/(TP+FP); Recall = TP/(TP+FN); F1 = 2TP/(2TP+FP+FN). TP and FN count detected and missed malicious URLs; FP and TN count incorrectly flagged and correctly classified benign URLs. Zero-denominator quality metrics are set to zero. Operational errors count execution failures, not classification mistakes.

6. Limitations and next steps

Domain-disjoint splitting prevents shared domains within a split, but campaigns or URL templates can still cross boundaries and repeats reuse observations. Labels and sampling limit generalisation to phishing. Fixed model settings do not constitute a tuned detector. The selected-feature timing path still calculates all 16 lexical features before selecting seven, so it does not measure the potential savings of an optimized extractor. Timing applies to the recorded Windows machine, local process queue and finite scheduled workload.

Future class-weight or decision-threshold tuning should use training/validation data and fresh held-out evaluation. The present recall shortfall is retained as a result. Neither thresholds nor test outcomes are altered to make guardrails pass.

7. Reproducibility and contributions

Recorded run: 20261105T122952Z_dbea27e6. Environment: Python 3.11.9, Windows build 22631, 12 logical CPUs; numerical thread limit one. Metadata records package versions, dataset/source hashes and split details; the environment lock records installed packages. The full raw log is losslessly compressed, while a four-record plain JSONL illustrates its schema. The notebook computes from the full log. Old batch results are clearly marked in archive/task3_batch/.

Zhanna prepared the original data, experiment, processing and figures. Dariyana reviewed the dataset/metric interpretation, revised documentation and notebook, ran the domain-disjoint RPS experiment and organized its results. The initial Task 3 notebook/domain/report upload was committed by Zhanna; subsequent corrections and the rerun are recorded in Dariyana's commits. Contribution descriptions distinguish the agreed division of work from Git upload authorship.

Repository: github.com/dariyanagabdylshukir-crypto/phishing-url-ml-comparison
Work branch: fix/task3-review. This report is based on the rerun preserved by commit 4a170c9; final analysis-module updates accompany this report. A final GitHub release must be created after the completed notebook and report are committed.